

Trabajo Práctico

Tópicos de Programación

Comisión: 04-3600

Miércoles - turno tarde

F1: Gestión de Temporada

Consigna

Desarrollar un sistema en lenguaje C que permita gestionar la información de una temporada de Fórmula 1, incluyendo pilotos, escuderías, carreras y estadísticas.

El sistema debe evolucionar a lo largo de la cursada, incorporando los contenidos de las distintas unidades de la materia según consideren necesarios y óptimos para la resolución.

El sistema deberá trabajar con los siguientes archivos:

- Archivos de texto (entrada inicial): pilotos.txt y escuderias.txt. Estos archivos se utilizan para la carga inicial de datos.
- Archivos binarios: pilotos.dat, escuderias.dat y carreras.dat. Se debe permitir altas, bajas lógicas (marcas registros como eliminados, estas también deben registrarse en un archivo de bajas) y modificaciones.

Tenga en cuenta que no se sabe exactamente cuántos pilotos hay en total.

Se deberán implementar funciones para búsqueda de pilotos, ordenamiento por distintos criterios y relación entre las distintas estructuras.

Funcionalidades mínimas:

1. Listar pilotos y sus puntos.
2. Registrar una carrera (ingreso de posiciones).
3. Calcular y actualizar puntos automáticamente.
4. Mostrar ranking de pilotos (ordenado) de la temporada en cuestión.
5. Mostrar pilotos por escudería.
6. Calcular estadísticas de pilotos.
7. Exportar datos a archivos de texto.

A partir del archivo de carreras, el sistema deberá generar estadísticas de cada piloto, calcular promedios, máximos y mínimos; y permitir consultas como top 5 pilotos con más victorias, piloto con mejor promedio, mejor posición o peor posición.

Puntos adicionales:

Para acceder a la promoción de la materia, el equipo deberá implementar al menos 2 de los siguientes puntos avanzados.

- A. *Índice persistente para acceso directo.* Implementar un archivo índice (.idx) que permita acceder a los pilotos de forma eficiente. El archivo debe estar ordenado por ID.
- B. *Motor de consultas genérico.* Implementar funciones genéricas.
- C. *Simulación de carrera,* generando resultados automáticamente.
- D. *Sistema incremental de estadísticas.* Optimizar el cálculo de estadísticas evitando computar desde cero.
- E. *Combinación de temporadas.* Implementar una funcionalidad que permita combinar dos archivos de carreras (carreras.dat) en uno solo, manteniendo el orden (por fecha o ID de carrera) y evitando duplicados.

El sistema deberá implementar los siguientes TDA:

Piloto:

CAMPO	TIPO DE DATO
id	unsigned
nombre	char[30]
nacionalidad	char[30]
id_escuderia	Unsigned
puntos_acumulados	unsigned
estado	Char (A:Activo, R:Retirado, S:Suspendido)
fechaNacimiento	unsigned long long

Escudería

CAMPO	TIPO DE DATO
id	unsigned
codigo	char [4]
nombre	char[30]
pais	char[50]
estado	Int (1: activo, 0: inactivo)

Carrera

CAMPO	TIPO DE DATO
id	int
circuito	char[20]
fecha	unsigned long long
int	estado
Cant_resultados	int
resultados	Matriz que contenga los resultados

Formato de Entrega:

Al momento de la entrega final,

- Todos y cada uno de los integrantes deben subir su TP en una carpeta **.zip** con el siguiente nombre:
 - **DNI_APELLIDO_NOMBRE_TP.zip**
- Deben **borrar** las carpetas **"bin"** y **"obj"**.
- Dentro del zip debe estar el proyecto con el "main" y las librerías que hayan creado y/o utilizado.
- El TP deberá ser entregado por la plataforma **MleL**, en la sección de **Portafolio**.

Condiciones Generales:

- Grupos de entre 4 y 5 integrantes, sin excepción.
- La fecha de entrega será informada por los docentes.
- El TP tiene una defensa oral, cuya nota es individual.

Requisitos de aprobación generales:

- Cumplimentar con la consigna completa.
- Crear los casos de uso necesarios y pertinentes.
- 0 errores 0 "warnings".
- Que funcione mínimo para todos los casos de prueba que presentan.
- Vectores y cadenas de texto deberán ser manipulados utilizando aritmética de punteros.
- Las soluciones tienen que ser eficientes:
 - En el uso de memoria, por tanto, no declare vectores o matrices auxiliares si no es necesario.
 - En el uso de archivos temporales o auxiliares.
 - En cantidad de ciclos de procesador y en el caso de matrices las soluciones deben ser óptimas.
 - Los recorridos de archivos tienen que ser mínimos.
- Nunca acceda a memoria que no le pertenece.
- No deje archivos abiertos.
- Declare variables al inicio del bloque y no utilice VLA (Variable length arrays).
- Se evaluará la prolijidad general:
 - Código prolijo, dividido en funciones,
 - Nombres descriptivos,
 - Nombres significativos de variables.
- Las soluciones tienen que estar desarrolladas en ANSI C para garantizar tanto como sea posible compatibilidad multiplataforma.
- Nunca mezcle funciones de manipulación de archivos de texto con funciones de operación de archivos binarios.